

Guia de Arquitetura: Microsserviços

Case de Estudo Técnico: Plataforma de E-commerce

1. Visão Geral e Stack Tecnológica

Este documento detalha a modelagem de um sistema de e-commerce baseado em microsserviços. A arquitetura foi desenhada focando em alta escalabilidade, desacoplamento e resiliência, ideal para suportar cenários de alto tráfego.

Para a interface de usuário (Frontend), a integração com a API Gateway pode ser implementada de forma reativa utilizando frameworks modernos como Angular ou React, garantindo o consumo eficiente dos serviços de backend.

2. Definição dos Microsserviços

Cada serviço possui domínio próprio e isolamento de dados, respeitando as melhores práticas de sistemas distribuídos.

Microsserviço	Responsabilidade	Banco de Dados
API Gateway	Roteamento unificado, rate limiting e proxy reverso.	-
Auth & Usuários	Gestão de perfis, endereços e autenticação via JWT.	PostgreSQL
Catálogo	Listagem de produtos, busca, categorias e estoque.	MongoDB
Pedidos (Checkout)	Gestão do carrinho, aplicação de regras de negócio.	PostgreSQL
Pagamentos	Comunicação com gateways, processamento de saldo.	PostgreSQL
Notificações	Worker para disparo assíncrono de emails e SMS.	-

3. Padrões de Comunicação

Síncrona (REST): Utilizada na comunicação externa, como as requisições do frontend (Angular/React) para o API Gateway e em consultas imediatas de leitura (ex: o cliente visualizando a página de um produto através do serviço de Catálogo).

Assíncrona (Mensageria com RabbitMQ): A espinha dorsal da arquitetura interna. Garante que os microsserviços funcionem de forma independente. Utiliza o padrão Publisher/Subscriber para evitar que uma lentidão em um serviço (como a operadora de cartão de crédito) impacte o resto da aplicação.

4. O Fluxo Assíncrono de Pedidos

Abaixo detalhamos o ciclo de vida de uma requisição de compra via RabbitMQ, desde a criação até a confirmação do pagamento.

Passo 1: Publicação do Evento (Producer)

O cliente finaliza a compra. O serviço de Pedidos cria o registro no PostgreSQL com o status AGUARDANDO_PAGAMENTO. Imediatamente, ele atua como Producer e publica um evento com o tópico pedido.criado na Exchange do RabbitMQ contendo o ID e os valores do pedido.

Passo 2: Roteamento (Exchange e Queue)

A Exchange do RabbitMQ recebe a mensagem e a encaminha em milissegundos para a fila configurada (ex: fila_pagamentos). A mensagem aguarda de forma segura na memória/disco.

Passo 3: Consumo e Processamento (Consumer)

O serviço de Pagamentos atua como Consumer. Ele detecta a mensagem, a consome da fila e inicia a transação no gateway de cartão de crédito. Se o serviço estiver em manutenção, não há perda de dados: as mensagens se acumulam na fila e são processadas quando o sistema normalizar.

Passo 4: Fechamento do Ciclo (Feedback)

Após finalizar a cobrança, o serviço de Pagamentos publica um novo evento no RabbitMQ: pagamento.aprovado (ou recusado). O serviço de Pedidos consome essa resposta e atualiza o banco de dados para o status PAGO. Ao mesmo tempo, o serviço de Notificações também escuta este evento para avisar o usuário por e-mail.

5. Dicas Práticas para o Desenvolvimento

- Configure sempre Dead Letter Queues (DLQ). Caso uma mensagem não consiga ser processada após algumas tentativas, ela deve ser movida para a DLQ para depuração manual, evitando travamentos nas filas principais.